

Function Approximation in Reinforcement Learning

From tabular values to deep policies
with quantum-state-preparation examples

FYS4411 – Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

Week 5

Learning goals for Week 5

By the end of the lecture, you should be able to:

- ▶ explain why tabular RL fails in large, continuous, or high-dimensional state spaces;
- ▶ write value-function approximation as $\hat{v}(s; w)$ and $\hat{q}(s, a; w)$;
- ▶ derive the basic squared-error and semi-gradient updates;
- ▶ explain the structure of deep Q-learning, target networks, and replay buffers;
- ▶ parametrize stochastic policies using neural networks;
- ▶ explain policy gradients, actor–critic methods, and PPO at a conceptual and mathematical level;
- ▶ formulate quantum state preparation as a deep-RL problem.

Roadmap

1. Why function approximation?
2. Value-function approximation
3. Deep Q-learning
4. Policy parametrization and policy gradients
5. Actor–critic methods and PPO
6. Quantum state preparation as a deep-RL application
7. Practical lessons and discussion questions

Why Week 5 matters

So far, our algorithms have mostly been tabular:

$$V(s), \quad Q(s, a).$$

This works when the state-action space is small enough that every value can be stored explicitly.

But physical systems rarely look like small tables:

- ▶ quantum states may be wavefunctions, density matrices, or measurement histories;
- ▶ molecular systems have high-dimensional continuous configurations;
- ▶ control systems and robots have continuous states and actions;
- ▶ agentic systems have text, tools, files, and long histories.

Takeaway. Function approximation is the move from storing values to learning functions.

From tabular RL to function approximation

In a finite tabular problem, a value function is a lookup table:

$$s \mapsto V(s), \quad (s, a) \mapsto Q(s, a).$$

A single update changes one entry.

With function approximation, we write

$$V(s) \rightsquigarrow \hat{v}(s; w), \quad Q(s, a) \rightsquigarrow \hat{q}(s, a; w).$$

The parameters w control many values at once.

Takeaway. The table disappears. The trainable function becomes the object being updated.

Why tables fail in physics

A state may be:

- ▶ a vector of continuous variables $(x, \dot{x}, \phi, \dot{\phi})$;
- ▶ a many-particle configuration $\mathbf{R} = (\mathbf{r}_1, \dots, \mathbf{r}_N)$;
- ▶ a wavefunction $|\psi\rangle$ or density matrix ρ ;
- ▶ a history of measurement outcomes $(o_0, o_1, \dots, o_t)$.

Even if one discretizes, the number of bins grows exponentially with dimension.

bins per dimension ^{d}

Takeaway. Function approximation is a response to the curse of dimensionality.

The central tradeoff

Function approximation gives generalization:

experience at one state can improve estimates at similar states.

But it also creates interference:

an update meant to fix one state may damage another.

Tabular

- ▶ local updates
- ▶ transparent
- ▶ no generalization
- ▶ impossible at scale

Function approximation

- ▶ shared parameters
- ▶ generalizes
- ▶ can be unstable
- ▶ works at scale

Neural networks as parametrized functions

A neural network is a parametrized map:

$$f_w : X \rightarrow Y.$$

In supervised learning, the target is fixed:

$$(x_i, y_i), \quad f_w(x_i) \approx y_i.$$

In reinforcement learning, the target is produced by interaction:

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

The same optimizer may be used, but the data-generating process is different.

Takeaway. Deep RL is neural-network training inside a feedback loop.

Approximating state values

For prediction under a policy π , the true value is

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

We approximate it by

$$\hat{v}(s; w).$$

The learning objective is to choose w such that

$$\hat{v}(s; w) \approx v_{\pi}(s).$$

The network takes the state as input and outputs one scalar.

$$s \longmapsto \hat{v}(s; w).$$

Approximating action values

For control, action values are often more useful:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

We approximate

$$q_{\pi}(s, a) \approx \hat{q}(s, a; w).$$

For discrete actions, a common network design is

$$f_w(s) = (\hat{q}(s, a_1; w), \dots, \hat{q}(s, a_m; w)).$$

Then the greedy action is

$$a^* \in \arg \max_{a \in \mathcal{A}} \hat{q}(s, a; w).$$

The supervised-looking loss

If the target value Y_t were known, we could minimize

$$L_t(w) = \frac{1}{2} [Y_t - \hat{v}(S_t; w)]^2.$$

The gradient direction is

$$[Y_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

Thus a generic value-function update is

$$w \leftarrow w + \alpha [Y_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

Takeaway. The main RL question is: what target should we use?

Monte Carlo targets with function approximation

Monte Carlo uses the complete return:

$$Y_t = G_t.$$

The loss becomes

$$L_t(w) = \frac{1}{2} [G_t - \hat{v}(S_t; w)]^2.$$

The update is

$$w \leftarrow w + \alpha [G_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

This is close to supervised regression, except the targets are sampled trajectory returns.

Takeaway. Monte Carlo approximation fits complete sampled outcomes.

Temporal-difference targets with function approximation

TD uses a bootstrapped one-step target:

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

The update becomes

$$w \leftarrow w + \alpha [R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

The bracketed term is the TD error:

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

Why “semi-gradient”?

The TD target

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w)$$

itself depends on w .

A full gradient of the squared error would differentiate through both:

$$Y_t \quad \text{and} \quad \hat{v}(S_t; w).$$

In standard TD learning, we treat Y_t as fixed during the update.

Thus the gradient is taken only through the current prediction:

$$\nabla_w \hat{v}(S_t; w).$$

Takeaway. Semi-gradient TD updates the prediction toward a target without differentiating through the target.

Monte Carlo

$$Y_t = G_t$$

- ▶ waits for full outcome
- ▶ no bootstrapping
- ▶ often high variance
- ▶ unbiased for sampled returns

TD

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w)$$

- ▶ updates immediately
- ▶ bootstraps
- ▶ often lower variance
- ▶ can be biased

Takeaway. Function approximation does not remove the MC/TD tradeoff; it makes it more consequential.

Action-value approximation and Q-learning

Approximate the optimal action-value function:

$$q_*(s, a) \approx \hat{q}(s, a; w).$$

For a transition $(S_t, A_t, R_{t+1}, S_{t+1})$, the Q-learning target is

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w).$$

The loss is

$$L_t(w) = \frac{1}{2} [Y_t - \hat{q}(S_t, A_t; w)]^2.$$

The update moves the network output for (S_t, A_t) toward the Bellman optimality target.

Deep Q-learning: the network view

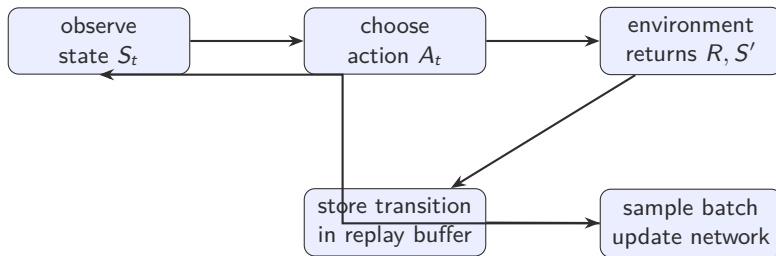
For discrete actions, the network often outputs all action values at once:

$$f_w(s) = \begin{bmatrix} \hat{q}(s, a_1; w) \\ \hat{q}(s, a_2; w) \\ \vdots \\ \hat{q}(s, a_m; w) \end{bmatrix}.$$

The agent selects actions using ϵ -greedy exploration:

$$A_t = \begin{cases} \arg \max_a \hat{q}(S_t, a; w), & \text{with probability } 1 - \epsilon, \\ \text{random action,} & \text{with probability } \epsilon. \end{cases}$$

Deep Q-learning loop



The Bellman target supplies the supervised-looking regression target for each sampled transition.

The moving-target problem

In deep Q-learning, the target

$$R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w)$$

uses the same parameters w being updated.

This creates a moving target:

as w changes, the label changes.

A common stabilization technique is a target network:

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w^-).$$

The online parameters w are updated often; the target parameters w^- are updated slowly.

Experience replay

Consecutive samples from a trajectory are highly correlated:

$$(S_t, A_t, R_{t+1}, S_{t+1}), \quad (S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}).$$

Replay buffers reduce this problem by storing transitions:

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N.$$

Updates are made from mini-batches:

$$B \sim \mathcal{D}.$$

This reuses experience and makes batches less temporally correlated.

Takeaway. Target networks stabilize the labels; replay buffers stabilize the data.

Mini-batch DQN loss

For a batch B , define

$$y(r, s') = \begin{cases} r, & s' \text{ terminal,} \\ r + \gamma \max_{a'} \hat{q}(s', a'; w^-), & \text{otherwise.} \end{cases}$$

Then minimize

$$L(w) = \frac{1}{|B|} \sum_{(s, a, r, s') \in B} \frac{1}{2} [y(r, s') - \hat{q}(s, a; w)]^2.$$

This is ordinary gradient-based optimization applied to Bellman targets.

What can go wrong?

Deep RL is unstable because several objects change at once:

- ▶ the value estimates change;
- ▶ the targets change;
- ▶ the policy changes;
- ▶ the data distribution changes;
- ▶ exploration changes what is observed;
- ▶ bootstrapping can amplify errors.

This is unlike supervised learning, where the dataset and labels are usually fixed.

Takeaway. Deep RL is not just supervised learning with a reward; it is supervised-looking optimization in a feedback loop.

When value-based methods fit

DQN-style methods are natural when:

- ▶ actions are discrete;
- ▶ the goal is to learn action values;
- ▶ $\arg \max_a Q(s, a)$ is easy;
- ▶ off-policy replay is useful;
- ▶ one wants sample reuse.

They are less natural when:

- ▶ actions are continuous;
- ▶ the policy must be stochastic by design;
- ▶ action constraints are complex;
- ▶ the maximum over actions is difficult.

Policy parametrization

Value-based methods learn values and derive a policy:

$$\pi(s) \in \arg \max_a \hat{q}(s, a; w).$$

Policy-based methods parametrize the policy directly:

$$\pi_{\theta}(a | s) = P_{\theta}(A_t = a | S_t = s).$$

The objective is

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_0].$$

The optimization problem is

$$\theta^* \in \arg \max_{\theta} J(\theta).$$

Takeaway. The policy itself becomes the trainable function.

Softmax policies for discrete actions

For finite action space

$$\mathcal{A} = \{a_1, \dots, a_m\},$$

a neural network can output logits

$$z_\theta(s, a_1), \dots, z_\theta(s, a_m).$$

The softmax policy is

$$\pi_\theta(a_i | s) = \frac{\exp(z_\theta(s, a_i))}{\sum_{j=1}^m \exp(z_\theta(s, a_j))}.$$

This guarantees

$$\pi_\theta(a_i | s) \geq 0, \quad \sum_i \pi_\theta(a_i | s) = 1.$$

Gaussian policies for continuous actions

For continuous control, the policy may output a distribution:

$$\pi_{\theta}(a \mid s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

The action used during training is sampled:

$$A_t \sim \pi_{\theta}(\cdot \mid S_t).$$

In evaluation, one may use the mean action:

$$A_t = \mu_{\theta}(S_t).$$

Continuous policies are important in quantum control because pulse amplitudes and fields are often continuous variables.

Trajectory objective

A finite trajectory is

$$\tau = (s_0, a_0, r_1, s_1, \dots, s_T).$$

The objective can be written as

$$J(\theta) = \sum_{\tau} P_{\pi_{\theta}}(\tau) G_0(\tau).$$

The dependence on θ enters through the policy factors:

$$P_{\pi_{\theta}}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

The log-derivative trick

Use

$$\nabla_{\theta} P_{\pi_{\theta}}(\tau) = P_{\pi_{\theta}}(\tau) \nabla_{\theta} \log P_{\pi_{\theta}}(\tau).$$

Then

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log P_{\pi_{\theta}}(\tau) G_0].$$

Because the environment probabilities do not depend on θ ,

$$\nabla_{\theta} \log P_{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Policy gradient update

A common form of the policy gradient is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t \right].$$

The stochastic gradient ascent update is

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

Actions followed by high return become more likely in similar states.

REINFORCE in words

The update

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t$$

means:

- ▶ if action A_t was followed by high return, increase its log-probability;
- ▶ if action A_t was followed by low return, decrease its log-probability;
- ▶ the policy changes directly, not through a separate greedy step.

The weakness is variance: complete returns can fluctuate greatly.

Baselines and advantages

Subtracting a baseline can reduce variance:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) [G_t - b(S_t)].$$

A natural baseline is the value function:

$$b(S_t) \approx v_{\pi_{\theta}}(S_t).$$

The difference

$$G_t - v_{\pi_{\theta}}(S_t)$$

measures whether the observed return was better or worse than expected.

Takeaway. This motivates actor-critic methods.

Actor–critic architecture

Actor

Policy network:

$$\pi_{\theta}(a \mid s)$$

Chooses actions.

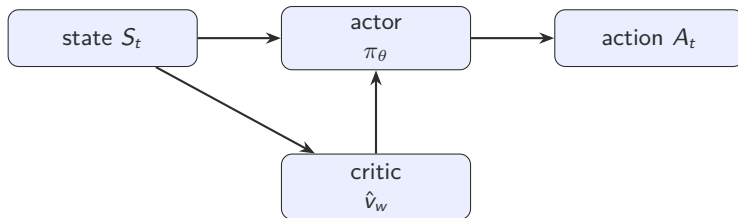
Critic

Value network:

$$\hat{v}(s; w)$$

Evaluates states or actions.

The actor controls behavior; the critic supplies a learning signal.



TD error as advantage estimate

A one-step TD error is

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

Interpretation:

- ▶ $\delta_t > 0$: the outcome was better than expected;
- ▶ $\delta_t < 0$: the outcome was worse than expected.

The actor update is

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

The critic update is

$$w \leftarrow w + \alpha_w \delta_t \nabla_w \hat{v}(S_t; w).$$

Why actor–critic matters

Actor–critic combines two advantages:

- ▶ policy gradients optimize stochastic or continuous policies directly;
- ▶ value learning provides lower-variance learning signals.

Compared with REINFORCE:

G_t is replaced by δ_t or an advantage estimate.

Compared with DQN:

the policy is trained directly rather than extracted by $\arg \max$.

Practical issue: policy updates can be too large

Suppose data were collected using an old policy

$$\pi_{\theta_{\text{old}}}.$$

After a large gradient step, the new policy

$$\pi_{\theta}$$

may behave very differently.

Then the data no longer represent the new policy well.

Takeaway. Modern policy-gradient methods often restrict how far the policy can move in one update.

PPO: probability ratio

PPO compares the new and old policy using

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

If $r_t(\theta) > 1$, the new policy assigns higher probability to the sampled action.

If $r_t(\theta) < 1$, it assigns lower probability.

The update is weighted by an advantage estimate $\hat{A}_t$.

PPO clipped objective

The unclipped surrogate objective is

$$L^{\text{PG}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right].$$

PPO clips the ratio:

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon).$$

The clipped objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Takeaway. PPO keeps the policy-gradient idea but limits overly aggressive updates.

PPO as actor–critic

PPO typically trains:

- ▶ an actor $\pi_{\theta}(a \mid s)$ using the clipped objective;
- ▶ a critic $\hat{v}(s; w)$ using a value loss.

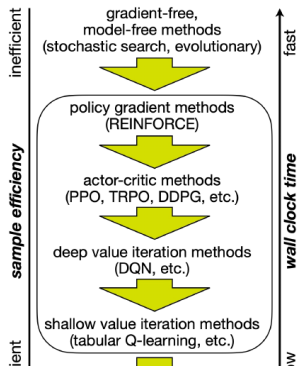
The critic loss may be

$$L^V(w) = \frac{1}{2} \mathbb{E}_t \left[(\hat{V}_t - \hat{v}(S_t; w))^2 \right].$$

The actor uses advantage estimates, often derived from the critic.

PPO is popular because it is relatively robust and works for many continuous-control tasks.

Algorithm placement



The rlqt review places common RL families along a rough sample-efficiency and runtime axis.

For Week 5, the important part is not the exact ranking, but the design logic:

- ▶ DQN: value function approximation;
- ▶ REINFORCE: direct policy gradients;
- ▶ PPO: actor–critic with conservative updates;
- ▶ model-based RL: use a learned or known model.

Source: *Bukov and Marquardt, Reinforcement Learning for Quantum Technology, Fig. 6.*

Choosing an algorithm

Problem feature	Natural method	Reason
Discrete actions	DQN / Q-learning	output one value per action
Continuous controls	actor-critic / PPO	parametrize action distribution
Sparse terminal reward	MC / policy gradient	learns from episode outcome
Long horizon	TD / critic	bootstrapped learning signal
Noisy measurements	stochastic policy / PPO	robust expectation-based updates
Partial observations	recurrent or belief-state policy	state is not directly observable

Quantum state preparation: the application

Quantum state preparation is a natural RL application because the agent chooses controls that manipulate a physical system.

The goal is to transfer an initial state to a target state in fixed time T :

$$|\psi(0)\rangle \longrightarrow |\psi_\star\rangle.$$

A time-dependent Hamiltonian contains the control protocol:

$$H(t) = H_0 + \sum_k u_k(t) H_k.$$

The control amplitudes $u_k(t)$ are the actions or are generated by the policy.

Source: rlqt, Sec. V, page 20.

Fidelity as figure of merit

The standard figure of merit is the fidelity

$$F = |\langle \psi_{\star} | \psi(T) \rangle|^2, \quad F \in [0, 1].$$

The time-evolved state is

$$|\psi(T)\rangle = \mathcal{T} \exp \left(-i \int_0^T H(t) dt \right) |\psi(0)\rangle.$$

High fidelity means the prepared state is close to the target.

Many studies instead minimize the infidelity:

$$1 - F.$$

Source: rlqt, page 20.

Mapping quantum control to RL

RL object	Quantum-control interpretation
State/observation	wavefunction, density matrix, observables, measurement record
Action	field amplitude, pulse phase, detuning, gate choice, measurement setting
Transition	Schrodinger, Lindblad, or measurement-update dynamics
Reward	fidelity, energy decrease, sensitivity, error suppression, constraint penalties
Policy	control protocol generator $\pi_{\theta}(a s)$

The same mathematical object π_{θ} may represent a pulse controller, a gate selector, or a feedback rule.

Partial observability in quantum systems

A central issue: quantum states are not directly measurable.

Possible RL states/observations:

- ▶ exact wavefunction or density matrix in simulation;
- ▶ tomographic approximation for small systems;
- ▶ expectation values of selected observables;
- ▶ measurement outcomes over time;
- ▶ belief-state or recurrent representation.

Thus many quantum-control problems are better described as partially observable MDPs.

Source: rlqt, page 20.

Why function approximation is essential in quantum control

A neural policy can map limited observations to controls:

$$o_t \longmapsto \pi_\theta(a_t \mid o_t).$$

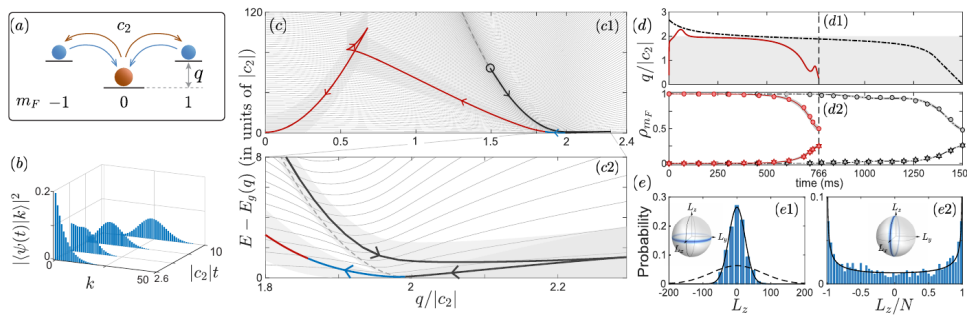
A neural critic can evaluate expected future success:

$$\hat{v}(o_t; w) \approx \mathbb{E}[G_t \mid o_t].$$

This matters because:

- ▶ observations may be continuous and noisy;
- ▶ actions may be bounded and continuous;
- ▶ rewards may be delayed until the final state;
- ▶ optimal protocols may be nonadiabatic and unintuitive.

Featured application: Dicke-state preparation



An RL agent prepares a Dicke state in a spin-1 Bose-Einstein condensate by learning a control protocol across a quantum phase transition.

Source: Bukov and Marquardt, *rlqt review*, Fig. 7 adapted from Guo et al.

Dicke-state preparation: physical setup

The system is a spinor BEC of ^{87}Rb atoms in the $F = 1$ hyperfine manifold:

$$m_F = 0, \pm 1.$$

The Hamiltonian is given as

$$\hat{H} = \frac{c_2}{2N} \hat{L}^2 - q(t) \hat{N}_0.$$

Here $q(t)$ is a controllable quadratic Zeeman shift.

The target is a balanced Dicke state:

$$|\psi_{\text{Dicke}}\rangle = |N, L_z = 0\rangle.$$

Source: rlqt, pages 20–21.

Why adiabatic preparation is difficult

The polar BEC state is easy to prepare initially:

$$|0, N, 0\rangle.$$

The Dicke state lies across a quantum critical point.

Near the critical point, the spectral gap closes, making slow adiabatic preparation difficult.

In the experiment described in the review:

- ▶ adiabatic timescales exceed coherence-time limitations;
- ▶ losses become problematic;
- ▶ the RL protocol identifies useful nonadiabatic dynamics.

Source: rlqt, page 20.

Dicke-state preparation as an RL problem

RL object	Dicke-state example
Observation	BEC density, fluctuations, first-order correction amplitude and phase
Action	continuous bounded choice of normalized Zeeman shift $q(t)/ c_2 \in [-3, 3]$
Policy	piecewise-constant control protocol generator
Reward	fidelity differences or experimentally accessible sensitivity increments
Algorithm	actor-critic PPO pipeline

The observations have a large- N limit, enabling pre-training at smaller sizes and deployment at larger sizes.

Source: rlqt, page 21.

Reward design in the Dicke example

The review describes a staged pipeline.

First, in exact dissipationless dynamics, use fidelity increments:

$$r_t = F_t - F_{t-1}, \quad F_t = |\langle \psi_{\text{Dicke}} | \psi(t) \rangle|^2.$$

Then, in dissipative simulation, use an experimentally accessible sensitivity measure:

$$r_t = f_t - f_{t-1}.$$

The ideal Dicke state has $f = 1$, while the initial polar BEC has $f = 0$.

Source: rlqt, page 21.

What did the policy learn?

The learned protocol is not simply a slow adiabatic ramp.

The review reports that the agent finds a fast protocol that uses excited-level dynamics.

Interpretation:

- ▶ cross the critical region in a controlled way;
- ▶ create a useful amount of excitation;
- ▶ attenuate or redirect excitation toward the target state;
- ▶ exploit dynamics that are hard to guess manually.

Takeaway. Function approximation can represent control strategies that are nontrivial, continuous, and history-dependent.

Source: rlqt, pages 21–22.

Why PPO fits the quantum example

The Dicke-state application has features that favor actor–critic PPO:

- ▶ bounded continuous controls $q(t)/|c_2|$;
- ▶ noisy or approximate observations;
- ▶ delayed, physically meaningful rewards;
- ▶ need for stable policy updates;
- ▶ possibility of pre-training in simulation.

PPO supplies:

- ▶ a stochastic actor for exploration;
- ▶ a critic for lower-variance learning signals;
- ▶ clipping to avoid overly aggressive updates.

From two-level control to many-body control

For a two-level system, a state may be represented by a small vector:

$$|\psi\rangle = (c_0, c_1).$$

For a many-body system, the Hilbert-space dimension grows rapidly.

This changes the RL design:

- ▶ exact states may be impossible to use as inputs;
- ▶ observables become the practical state representation;
- ▶ reward may be indirect, e.g., energy or sensitivity;
- ▶ transfer across system sizes becomes important.

Takeaway. Function approximation is not optional in many-body quantum control.

Design pattern: quantum-state-preparation environment

A simple environment template:

S_t := observables or state representation,

A_t := control amplitude, detuning, or pulse parameter,

S_{t+1} := quantum evolution under $H(t)$,

R_{t+1} := fidelity improvement or physical proxy.

At the end of the episode:

$$R_T \sim F \quad \text{or} \quad -(1 - F).$$

For constraints, add penalties:

$$R_T = F - \lambda \sum_t |u_t|^2 - \eta \sum_t |u_t - u_{t-1}|^2.$$

Function approximation and physical constraints

A good deep-RL formulation should respect physics:

- ▶ bounded control amplitudes;
- ▶ bandwidth or smoothness constraints;
- ▶ measurement limits;
- ▶ decoherence and noise;
- ▶ hardware-specific action spaces;
- ▶ meaningful reward scales.

Function approximation can help, but it can also exploit simulator artifacts.

Takeaway. A neural policy is not automatically physical; the environment and reward must encode constraints.

RL versus optimal control

RL and optimal control share a control objective, but differ in emphasis.

Conventional optimal control	Reinforcement learning
often assumes differentiable model	can be model-free
optimizes one protocol	learns reusable policy
usually re-optimizes when conditions change	may generalize after training
strong when model is accurate	useful with uncertainty and measurements
can be more sample efficient	often more data hungry

Source: rlqt discussion of RL and quantum optimal control, page 19.

Interactive exercise: formulate a two-level problem

Consider

$$H(t) = \frac{\omega}{2}\sigma_z + \frac{u_t}{2}\sigma_x.$$

Goal:

$$|0\rangle \rightarrow |1\rangle \quad \text{in time } T.$$

In pairs, define:

- ▶ state or observation;
- ▶ action space;
- ▶ reward;
- ▶ episode termination;
- ▶ suitable algorithm: DQN, actor–critic, or PPO.

Discussion: when is function approximation enough?

Function approximation solves representation, but not everything.

Questions:

- ▶ Does the observation contain enough information?
- ▶ Is the reward aligned with the physical goal?
- ▶ Does the policy generalize or memorize simulator quirks?
- ▶ Is the learned protocol interpretable?
- ▶ Is the comparison to standard optimal control fair?

Practical checklist for deep RL in physics

Before training, specify:

- ▶ state/observation representation;
- ▶ action constraints;
- ▶ reward scale and penalties;
- ▶ simulator fidelity and noise model;
- ▶ baseline method;
- ▶ computational budget;
- ▶ evaluation protocol;
- ▶ physical interpretability checks.

Takeaway. Deep RL is useful only when the learned policy is compared against transparent physical baselines.

Summary: the mathematical arc

Tabular values: $V(s), Q(s, a)$

Value approximation: $\hat{v}(s; w), \hat{q}(s, a; w)$

TD target: $R_{t+1} + \gamma \hat{v}(S_{t+1}; w)$

DQN target: $R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w^-)$

Policy approximation: $\pi_{\theta}(a | s)$

Actor-critic: $\pi_{\theta}(a | s)$ plus $\hat{v}(s; w)$

PPO: actor-critic with clipped policy updates.

Summary: the conceptual arc

- ▶ Function approximation replaces lookup tables by trainable functions.
- ▶ Neural networks allow generalization across large or continuous state spaces.
- ▶ The price is instability: changing parameters changes many predictions.
- ▶ DQN stabilizes value learning with target networks and replay.
- ▶ Policy-gradient methods train the policy directly.
- ▶ Actor-critic methods use a critic to reduce variance.
- ▶ PPO limits policy movement to make updates more stable.
- ▶ Quantum state preparation is a natural application because it is sequential control under uncertainty.

References and pointers

Main lecture source:

- ▶ FYS4411 notes, Chapter 3: function approximation and deep reinforcement learning.

Quantum application source:

- ▶ Marin Bukov and Florian Marquardt, *Reinforcement Learning for Quantum Technology*, arXiv:2601.18953.
- ▶ Page 20: quantum state preparation, fidelity, and partial observability.
- ▶ Pages 20–22: RL preparation of Dicke states in a spinor BEC.

For next time:

- ▶ How do uncertainty and Bayesian thinking modify exploration?
- ▶ When should a physical system be modeled, simulated, or learned directly?